

DevOps and Automations TASK – 3

1.Install Terraform on your system and use it to write a basic configuration file main.tf that provisions a single AWS EC2 instance named 'music-streaming-server'.

Ans :

- Verify Terraform is installed
- Open PowerShell/Terminal and run:
- terraform --version
- You should see the Terraform version.
- Create a Terraform folder
- mkdir music-streaming-server
- cd music-streaming-server
- Create main.tf
- On Windows PowerShell:
- notepad main.tf
- Click Yes if Windows asks whether you want to create the file.
- Copy this into main.tf
- terraform {
- required_providers {
- aws = {
- source = "hashicorp/aws"
- version = "~> 6.0"
- }
- }
- }
- }
- provider "aws" {
- region = "us-east-1"

- }
- resource "aws_instance" "music_streaming_server" {
- ami = "ami-0c02fb55956c7d316"
- instance_type = "t2.micro"
- tags = {
- Name = "music-streaming-server"
- }
- }
- Save the file.
- Initialize Terraform
- terraform init
- Format the configuration
- terraform fmt
- Validate the configuration
- terraform validate
- You should see:
- Success! The configuration is valid.
- Preview the EC2 instance
- terraform plan
- Check that Terraform plans to create:
- aws_instance.music_streaming_server
- Create the EC2 instance
- terraform apply
- When prompted:
- Do you want to perform these actions?
- Enter:
- yes

- Verify in AWS
- Open the AWS EC2 Console and select Instances.
- You should find an instance with:
- Name: music-streaming-server
- Instance type: t2.micro
- Region: us-east-1
- State: Running

**2.Refactor your main.tf by splitting the EC2 resource into a separate file called ec2.tf and use a variables.tf file to define the instance type and AMI as input variables.

Hint: This helps make your configuration modular and reusable.**

Ans :

- Create variables.tf
- Create the file:
- notepad variables.tf
- Paste:
- variable "instance_type" {
- description = "EC2 instance type"
- type = string
- default = "t2.micro"
- }
- variable "ami_id" {
- description = "AMI ID for the EC2 instance"
- type = string
- default = "ami-0c02fb55956c7d316"
- }
- Save and close it.

- Create ec2.tf
- Create the file:
- notepad ec2.tf
- Paste:
- resource "aws_instance" "music_streaming_server" {
- ami = var.ami_id
- instance_type = var.instance_type

- tags = {
- Name = "music-streaming-server"
- }
- }

- Save and close it.
- Modify main.tf
- Open:
- notepad main.tf
- Replace its contents with:
- terraform {
- required_providers {
- aws = {
- source = "hashicorp/aws"
- version = "~> 6.0"
- }
- }
- }

- provider "aws" {
- region = "us-east-1"
- }

- Save and close it.
- Your final files
- main.tf
- terraform {
- required_providers {
- aws = {
- source = "hashicorp/aws"
- version = "~> 6.0"
- }
- }
- }

- provider "aws" {
- region = "us-east-1"
- }
- variables.tf
- variable "instance_type" {
- description = "EC2 instance type"
- type = string
- default = "t2.micro"
- }

- variable "ami_id" {
- description = "AMI ID for the EC2 instance"
- type = string
- default = "ami-0c02fb55956c7d316"
- }
- ec2.tf
- resource "aws_instance" "music_streaming_server" {

- `ami = var.ami_id`
- `instance_type = var.instance_type`
- `tags = {`
- `Name = "music-streaming-server"`
- `}`
- `}`
- Format the files
- `terraform fmt`
- Initialize Terraform
- `terraform init`
- Validate
- `terraform validate`
- Expected:
- Success! The configuration is valid.

3.Add a Terraform resource to your configuration that provisions an AWS RDS MySQL database instance named 'playlist-db' in a separate db.tf file, and use outputs.tf to display the database endpoint after apply.

Ans :

- Create db.tf
- Run:
- `notepad db.tf`
- Paste:
- `resource "aws_db_instance" "playlist_db" {`
- `identifier = "playlist-db"`
- `engine = "mysql"`
- `engine_version = "8.0"`
- `instance_class = "db.t3.micro"`

- `allocated_storage = 20`
- `db_name = "playlistdb"`
- `username = "admin"`
- `password = "ChangeMe123!"`
- `skip_final_snapshot = true`
- `tags = {`
- `Name = "playlist-db"`
- `}`
- `}`
- Save and close the file.
- Create `outputs.tf`
- Run:
- `notepad outputs.tf`
- Paste:
- `output "database_endpoint" {`
- `description = "RDS MySQL database endpoint"`
- `value      = aws_db_instance.playlist_db.endpoint`
- `}`
- Save and close the file.
- Format the configuration
- `terraform fmt`
- Validate
- `terraform validate`

4. Initialize a new Git repository in your Terraform project folder, commit all your configuration files, and write a README.md that explains how to deploy your infrastructure using

**Terraform.

Hint: Include setup steps and any variable values that need to be provided.**

Ans :

- Open PowerShell in your project folder
- cd C:\music-streaming-server
- Initialize Git
- git init
- Create .gitignore
- This prevents Terraform state and sensitive files from being committed.
- notepad .gitignore
- Add:
- .terraform/
- *.tfstate
- *.tfstate.*
- *.tfvars
- *.tfvars.json
- crash.log
- crash.*.log
- Save and close.
- Create README.md
- notepad README.md
- Copy this into the file:
- # Music Streaming Server Infrastructure
- This project uses Terraform to provision AWS infrastructure for a music streaming application.
- ## Infrastructure

- The Terraform configuration creates:
 - AWS EC2 instance named `music-streaming-server`
 - AWS RDS MySQL database named `playlist-db`
 - RDS database endpoint as a Terraform output

- `## Prerequisites`

- Install:
 - Terraform
 - Git
 - AWS CLI

- Configure AWS credentials before running Terraform.

- Verify Terraform:
 - ```bash`
 - `terraform --version`
 - Verify AWS:
 - `aws --version`

5. List three best practices for managing Terraform configurations in a real-world team project, and briefly explain how each practice

helps avoid problems when multiple developers are working together.

Ans :

- Use Remote State Storage
 - Store Terraform state in a shared remote backend such as Amazon S3 instead of keeping terraform.tfstate only on a developer's computer.
 - This gives the team a single source of truth and helps prevent developers from overwriting each other's state.
- Use Git and Pull Requests
 - Keep Terraform files in Git and require developers to submit changes through pull requests and code reviews.
 - This provides version history, allows teammates to review changes before deployment, and reduces accidental configuration changes.
- Use Separate Variable/Environment Configurations
 - Keep reusable settings in variables.tf and provide environment-specific values through .tfvars files or approved environment variables.
 - This prevents developers from hardcoding credentials, AMI IDs, instance sizes, or environment-specific settings and makes the same configuration easier to use across development, testing, and production.

